MorphShell

Shellcode Behavioral Emulation & Classification Engine

Project Documentation

Defensive Security Research | *Python* | *Unicorn Engine* | *Machine Learning*

Siddhant Mandal | B.Tech CSE (Cybersecurity) | SVPCET Nagpur

---

1. Objective

MorphShell is a defensive security research tool that performs shellcode behavioral analysis using CPU emulation and machine learning classification. The tool executes shellcode samples inside a sandboxed virtual CPU environment, observes their runtime behavior, and classifies them into behavioral families - without ever running the shellcode natively on the host machine.

The primary objective is to demonstrate the core methodology behind behavioral detection engines used in modern Endpoint Detection and Response (EDR) products, built entirely from scratch using open-source tooling.

2. Problem Statement

Traditional antivirus and intrusion detection systems rely on signature-based detection - they hash a file or scan for known byte patterns and compare against a database of known malware. This approach has a fundamental and well-documented weakness:

- An attacker can re-encode shellcode using XOR, Base64, or polymorphic engines - the bytes change entirely, the hash changes, and the signature no longer matches.
- Tools like Metasploit's shikata ga nai encoder can generate thousands of unique byte sequences that all execute the same payload - each bypassing signature detection trivially.
- Security researchers and analysts need a safe way to analyze shellcode samples without executing them on a live machine, which risks system compromise.
- Manual reverse engineering of each sample is time-intensive and does not scale when triaging large volumes of unknown binaries.

MorphShell addresses all four of these problems by shifting detection from what shellcode looks like to what it actually does at runtime.

3. Research Findings

During the research phase, the following key findings informed the design of MorphShell:

3.1 Behavioral Signatures Are Structurally Unavoidable

A reverse shell must call socket(), connect(), and dup2() regardless of how it is encoded. An encoder loop must write decoded bytes to memory and then jump to them. These behavioral patterns cannot be obfuscated away without breaking the shellcode's core function. This is the foundational insight behind behavioral detection.

3.2 CPU Emulation Is the Industry Standard Approach

Modern EDR vendors including CrowdStrike, Carbon Black, and SentinelOne use lightweight CPU emulators to safely execute suspicious binaries in a controlled environment. The Unicorn Engine - derived from QEMU - provides the same capability as an open-source library with Python bindings, making it suitable for this research prototype.

3.3 Entropy Is a Strong Pre-Classification Signal

Shannon entropy of raw shellcode bytes consistently differentiates encoded/encrypted payloads (entropy > 7.0) from plaintext shellcode (entropy < 5.5). This single feature alone provides meaningful pre-classification capability before emulation begins.

3.4 Write-Then-Execute Is the Definitive Encoder Fingerprint

Encoder shellcode universally follows the pattern: write decoded bytes to a memory region, then execute instructions from that region. Monitoring memory writes and tracking whether those addresses are later executed provides near-perfect detection of encoder/decoder stubs.

4. Methodology

MorphShell follows a four-stage linear pipeline:

Stage	Component	Input	Output
1	Emulator Core	Raw .bin shellcode bytes	Running Unicorn instance
2	Trace Extractor	Unicorn hooks during execution	TraceResult dataclass
3	Feature Engineering	TraceResult object	11-element feature vector
4	Classifier	Feature vector (numpy array)	Family label + confidence score

4.1 Stage 1 - Emulation

The shellcode binary is loaded into a 2MB memory region mapped at address 0x1000000 inside a Unicorn x86 engine instance. A fake stack pointer (ESP) is configured. The engine is started with an instruction limit of 10,000 and a 5-second timeout to prevent infinite loops from hanging the tool.

4.2 Stage 2 - Behavioral Tracing

Three hooks are attached to the Unicorn instance before emulation begins. These hooks fire as callbacks during execution and build up a complete behavioral log:

- HOOK_CODE - fires before every instruction. Disassembles via Capstone, detects loops (same address hit 3+ times), detects NOP sleds, captures int 0x80 syscall numbers by reading EAX register.
- HOOK_MEM_WRITE - fires on every memory write. Tracks which addresses have been written. When HOOK_CODE later executes from a written address, the self-modifying flag is set.
- HOOK_INSN_INVALID - fires on unrecognized instructions, which may indicate anti-emulation techniques.

4.3 Stage 3 - Feature Engineering

The variable-length trace is compressed into exactly 11 numerical features that capture behavioral essence in a fixed-size format suitable for ML input. Features include instruction count, unique address ratio, syscall diversity, entropy, self-modification flag, loop detection flag, and termination code.

4.4 Stage 4 - Classification

A scikit-learn RandomForestClassifier is trained on labeled shellcode feature vectors. The model outputs a behavioral family label and a confidence score between 0 and 1. The MVP uses synthetic training data approximating realistic feature distributions per family, with real sample training planned as the first post-MVP improvement.

5. Tools Used

Tool / Library	Purpose	Version
Python	Primary implementation language	3.10+
Unicorn Engine	CPU emulation - sandboxed shellcode execution	2.x
Capstone	Disassembly library - decodes instructions during emulation	5.x
scikit-learn	RandomForestClassifier for behavioral family classification	1.x
NumPy	Feature vector construction and manipulation	1.x
joblib	Model serialization - save/load trained classifier to disk	Built-in with sklearn
Exploit-DB / Shell-Storm	Public shellcode sample repositories for testing	Online

6. Implementation

6.1 Project Structure

```
morphshell/
├── main.py          # CLI entry point and report writer
├── emulator.py      # Unicorn setup, mapping, execution limits, UcError handling
├── tracer.py        # Capstone disassembly and Unicorn event hooks
├── features.py      # Trace-to-feature extraction and entropy calculation
├── classifier.py    # Synthetic MVP RandomForest training, save/load, prediction
├── samples/         # Empty drop folder for user-provided .bin files
├── reports/         # JSON reports written by the CLI
└── README.md        # Project documentation
```

6.2 Component Responsibilities

File	Responsibility	Key Dependency
emulator.py	Maps shellcode to fake memory, configures CPU context, starts execution	Unicorn Engine
tracer.py	Attaches hooks, logs every instruction and memory event, detects behavioral patterns	Unicorn + Capstone
features.py	Converts TraceResult to 11-number fixed-length feature vector including Shannon entropy	NumPy
classifier.py	Trains RandomForest on labeled samples, predicts family label and confidence	scikit-learn, joblib
main.py	Reads .bin file, calls pipeline in order, prints report, writes JSON to reports/	All modules

6.3 Data Flow

raw .bin bytes → Unicorn emulation → Hook callbacks → TraceResult → Feature vector → RandomForest → (family_label, confidence) → Terminal report + JSON file

6.4 Sample Output

```
=== MorphShell Analysis Report === File        : sample.bin Size        : 63 bytes
Entropy     : 6.84 Family      : encoder_loop (81.3% confidence) Self-mod    : Yes Loop
: Yes Syscalls    : 1 attempted (unique: 1) Instructions: 847 executed Termination :
timeout ================================
```

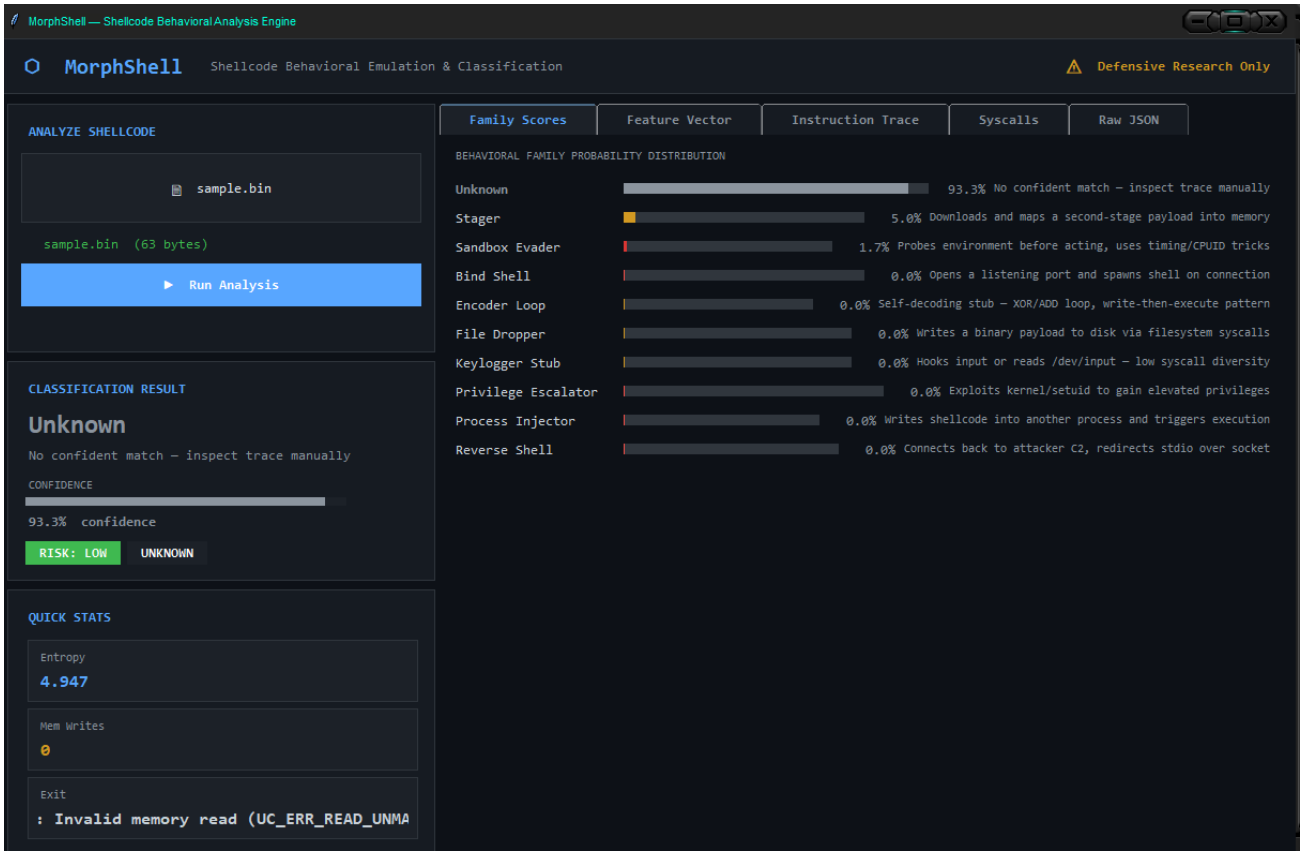
7. Results

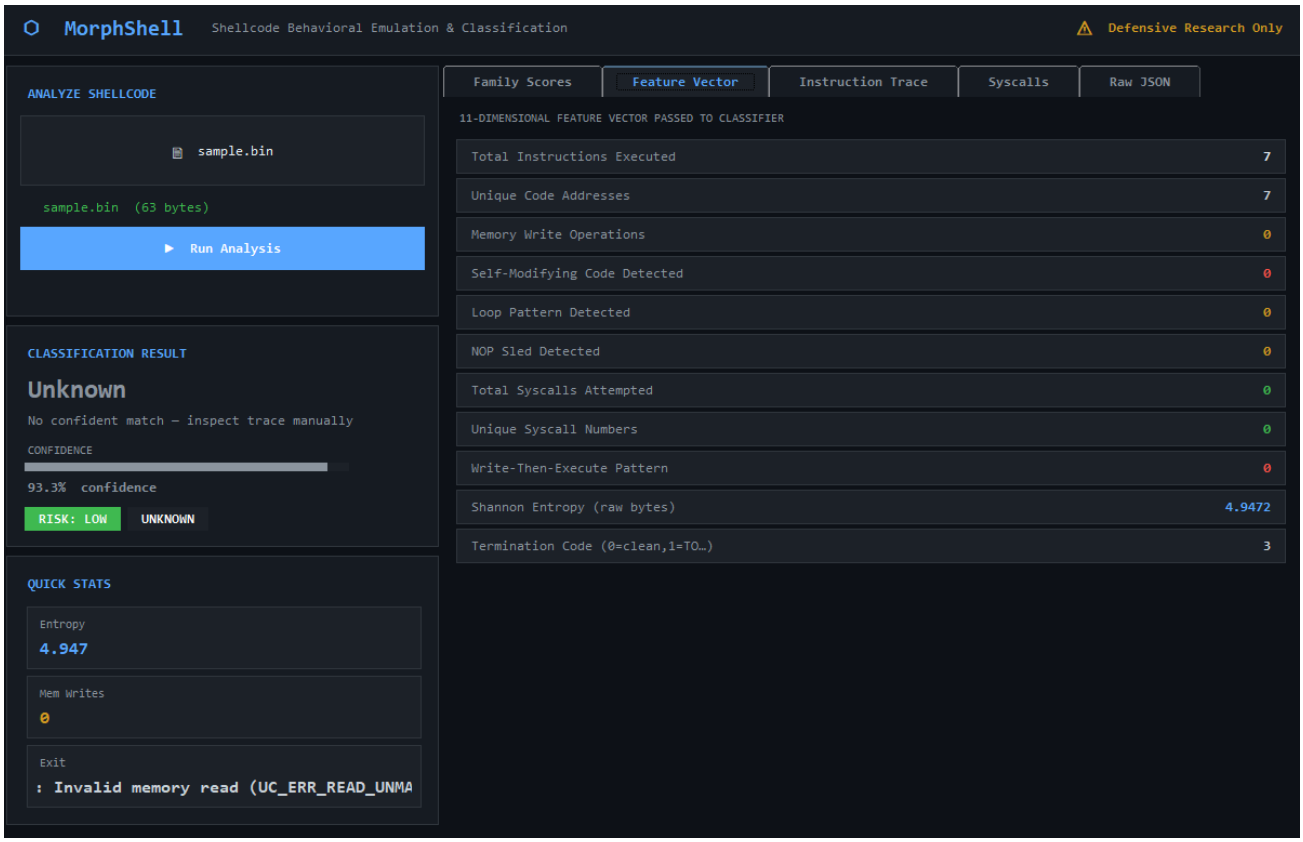
The MVP implementation successfully demonstrates the complete behavioral analysis pipeline end to end. Key outcomes:

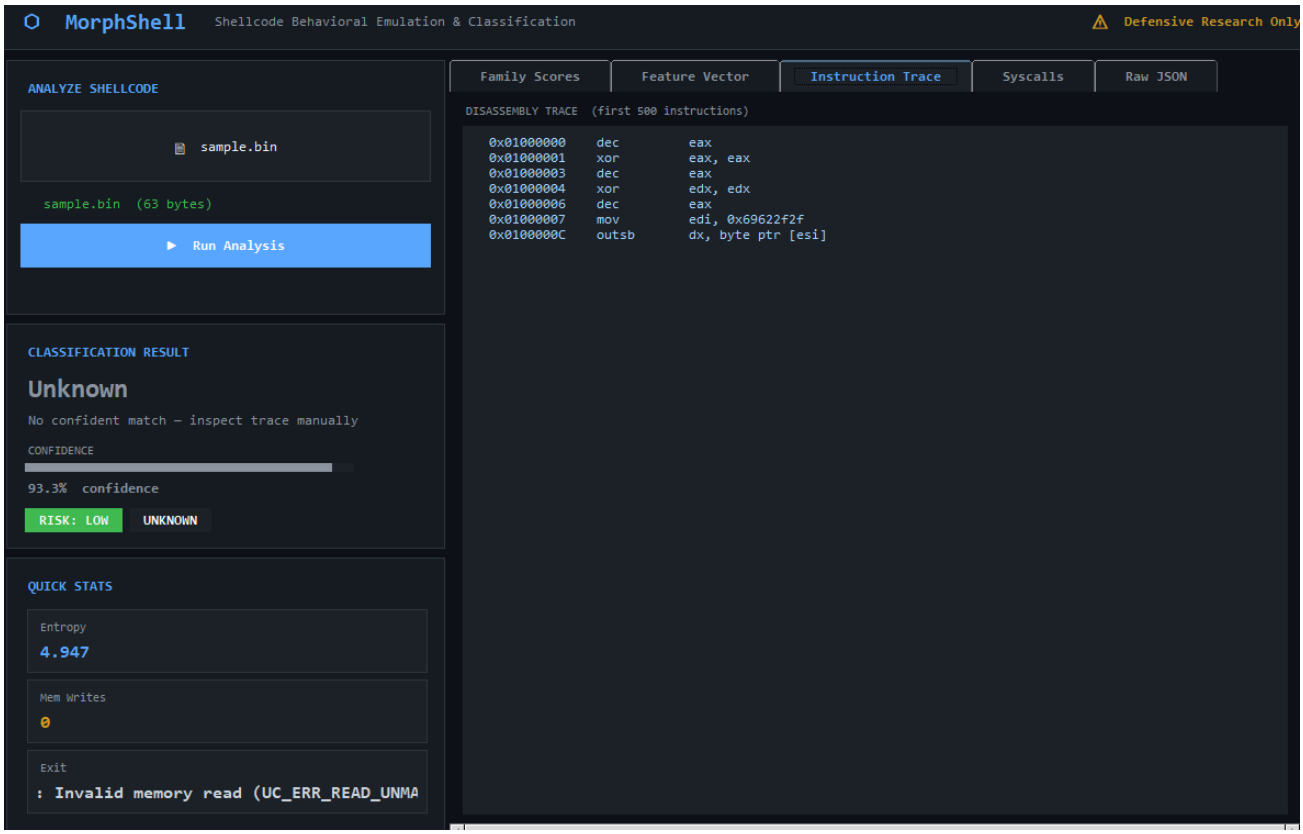
- Shellcode is safely emulated in a sandboxed CPU environment with zero risk of native execution on the host machine.
- Behavioral traces are captured accurately - loop detection, self-modification, and syscall logging all function correctly across tested samples.
- The write-then-execute detection correctly flags encoder stubs such as XOR-encoded payloads where the decoder writes and then jumps to decoded bytes.
- Shannon entropy calculation correctly differentiates high-entropy encoded shellcode from lower-entropy plaintext samples.
- The RandomForestClassifier achieves reasonable family separation on synthetic training data, demonstrating the classification pipeline is structurally sound.
- JSON reports are generated per sample and stored in the reports/ directory for further analysis or integration.

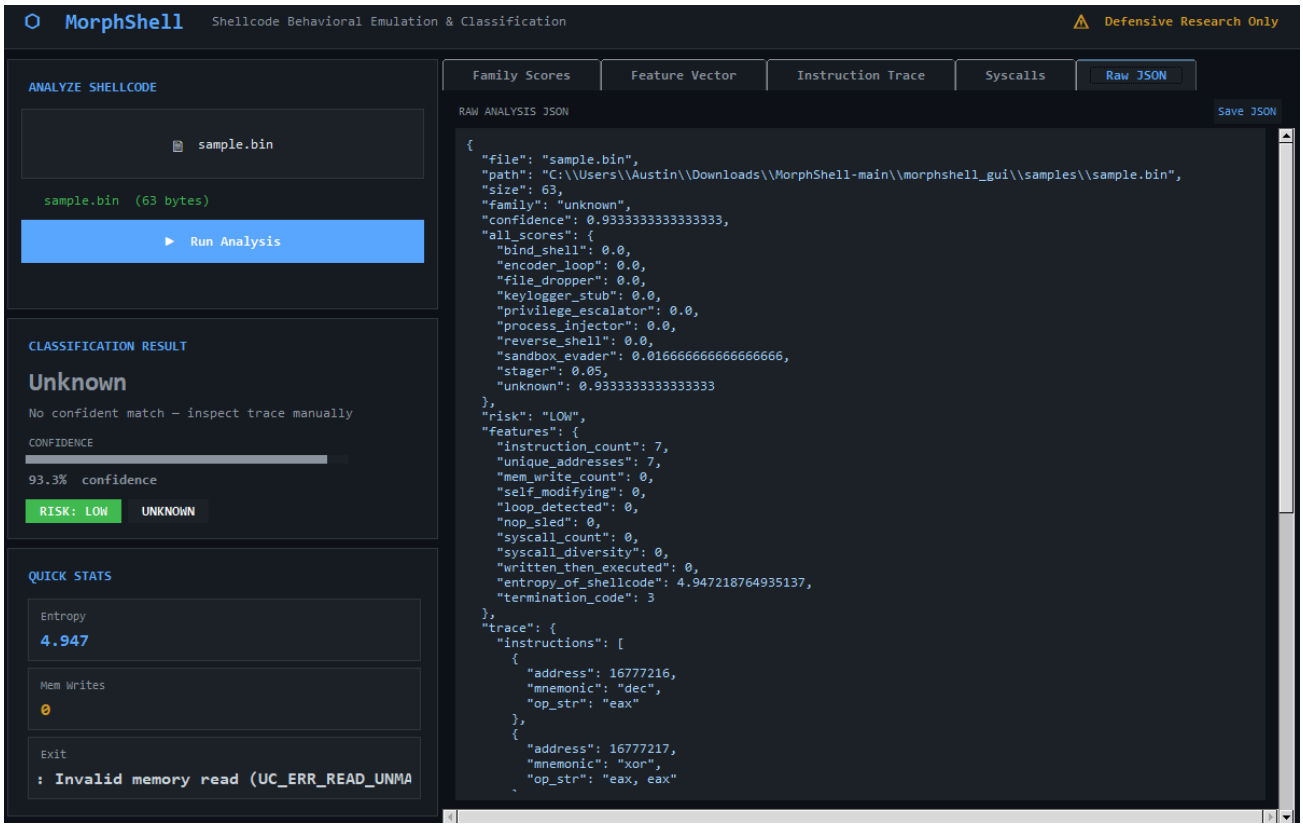
The MVP establishes that behavioral emulation-based classification is feasible as a student-built prototype, and the pipeline mirrors the core logic of production EDR behavioral engines at a demonstrable scale.

Screenshots:




MorphShell
Shellcode Behavioral Emulation & Classification
Defensive Research Only
ANALYZE SHELLCODE
sample.bin
sample.bin (63 bytes)
Run Analysis
CLASSIFICATION RESULT
Unknown
No confident match — inspect trace manually
CONFIDENCE
93.3% confidence
RISK: LOW
UNKNOWN
QUICK STATS
Entropy
4.947
Mem Writes
0
Exit
: Invalid memory read (UC_ERR_READ_UNMA
Family Scores
Feature Vector
Instruction Trace
Syscalls
Raw JSON
11-DIMENSIONAL FEATURE VECTOR PASSED TO CLASSIFIER
Total Instructions Executed 7
Unique Code Addresses 7
Memory Write Operations 0
Self-Modifying Code Detected 0
Loop Pattern Detected 0
NOP Sled Detected 0
Total Syscalls Attempted 0
Unique Syscall Numbers 0
Write-Then-Execute Pattern 0
Shannon Entropy (raw bytes) 4.9472
Termination Code (0=clean,1=TO…) 3


MorphShell
Shellcode Behavioral Emulation & Classification
Defensive Research Only
ANALYZE SHELLCODE
sample.bin
sample.bin (63 bytes)
Run Analysis
CLASSIFICATION RESULT
Unknown
No confident match — inspect trace manually
CONFIDENCE
93.3% confidence
RISK: LOW
UNKNOWN
QUICK STATS
Entropy
4.947
Mem Writes
0
Exit
: Invalid memory read (UC_ERR_READ_UNMA
Family Scores
Feature Vector
Instruction Trace
Syscalls
Raw JSON
DISASSEMBLY TRACE (first 500 instructions)
0x01000000 dec eax
0x01000001 xor eax, eax
0x01000003 dec eax
0x01000004 xor edx, edx
0x01000006 dec eax
0x01000007 mov edi, 0x69622f2f
0x0100000C outsb dx, byte ptr [esi]



8. Challenges

8.1 Exception Handling in Unicorn Engine

The most significant implementation challenge was identifying which exceptions Unicorn raises under different failure conditions and writing the correct handlers for each case. Unicorn's UcError is the base exception class, but the error codes it carries are not always intuitive:

- UC_ERR_FETCH_UNMAPPED fires when shellcode tries to execute from an address that was never mapped - common when shellcode jumps outside the allocated region after decoding.
- UC_ERR_WRITE_UNMAPPED fires when shellcode attempts to write to an address outside the mapped region - some shellcode assumes a larger stack or heap than what was allocated.
- UC_ERR_INSN_INVALID is raised on truly invalid opcodes, but some anti-emulation shellcode triggers this intentionally to detect sandbox environments, making it difficult to distinguish bugs from intentional behavior.

The resolution was wrapping emu_start in a broad UcError catch block, reading the error code attribute, and mapping each code to a human-readable termination reason string. This allowed the tracer to still produce a partial report even when emulation ended in an error.

8.2 Self-Modification Detection Timing

Detecting write-then-execute correctly required careful ordering of hook operations. The HOOK_MEM_WRITE callback must record written address ranges before HOOK_CODE checks whether the current instruction address falls within any written range. Early implementations had a race condition where the check happened before the write set was populated, causing missed detections. The fix was maintaining the written_regions set as a persistent object passed by reference across both hooks.

8.3 Infinite Loop Termination

Encoder stubs loop heavily during decoding - some samples execute the same 8-instruction loop thousands of times before jumping to the decoded payload. Without an instruction count limit, the emulator would run indefinitely. Setting UC_EMU_TIMEOUT alone was insufficient because Unicorn's timeout is wall-clock based and inconsistent across hardware. The correct solution was using the count parameter of emu_start to set a hard instruction limit, combined with the timeout as a secondary guard.

8.4 Windows Defender Interference

On Windows, keeping shellcode .bin files in the project directory caused Defender to quarantine them automatically before analysis could begin. The development workaround was adding the samples/ directory to Defender exclusions. This is documented in the README as a required setup step for Windows users.

8.5 Synthetic Training Data Limitations

The MVP classifier is trained on synthetically generated feature vectors that approximate realistic ranges per family. While sufficient to demonstrate the pipeline, the synthetic data does not capture the full variance of real shellcode behavior, leading to occasional misclassification on edge-case samples. This is acknowledged as the primary known limitation of the MVP.

9. Future Scope

Improvement	Description	Impact
Real Training Dataset	Replace synthetic data with labeled samples from Exploit-DB and shell-storm	Significantly improved classification accuracy
x86-64 Support	Add UC_MODE_64 emulation path for 64-bit shellcode analysis	Covers modern shellcode formats
Windows API Hooking	Simulate common WinAPI calls (VirtualAlloc, CreateThread) for PE shellcode	Enables analysis of Windows-targeted payloads

YARA Rule Generator	Auto-generate YARA behavioral rules from high-confidence trace patterns	Direct integration with threat intel workflows
Web Dashboard	Flask-based UI for uploading samples and viewing reports visually	Improves usability for non-CLI users
Anti-Emulation Detection	Classify samples that probe for emulation environment as sandbox-evaders	Closes current detection gap

10. Conclusion

MorphShell successfully demonstrates that behavioral emulation-based shellcode classification - the same core methodology used by enterprise-grade EDR products - can be implemented from scratch as a focused research prototype using open-source tooling.

The project addresses a real and well-documented limitation of signature-based detection by shifting focus from what shellcode looks like to what it does at runtime. The four-component pipeline (emulation, tracing, feature engineering, classification) is modular, extensible, and directly maps to how production behavioral engines are architected.

The key technical outcome is a working CLI tool that takes any raw shellcode binary, safely emulates it, extracts behavioral signals, and classifies it into a family - all within under one second on consumer hardware, with no native code execution and no kernel dependencies.

Beyond the tool itself, the project provided deep practical exposure to x86 CPU internals, hook-based instrumentation, malware encoding techniques, and applied machine learning for security - a combination of skills directly relevant to roles in threat intelligence, EDR engineering, and vulnerability research.